对以图计算为基础的 VSAG 向量库研究

谢宝玛 1120233506

(北京理工大学计算机学院 08012301 班)

摘要：虽然目前的“基于图的搜索算法”（比如著名的 HNSW）已经很快了，但在实际工业级应用中，仍面临三个大麻烦：内存访问乱，计算开销大，调优太痛苦 。VSAG 向量库针对上述问题提出了三个关键优化方向：内存优化，自动调优，计算加速。性能对比： 相比于目前工业界的标准库 HNSWlib，VSAG 在相同精度下，速度提升了高达 4 倍。落地情况： 已经在蚂蚁集团的大规模业务中实际部署应用。

关键词：vsag 库；内存优化；距离计算；自动调优

Research on the VSAG Vector Database Based on Graph Computing

Xie Baoma 1120233506
(School of Computer Science, Beijing Institute of Technology, Class 08012301)

Abstract: Although current *graph-based search algorithms* (such as the well-known HNSW) are already very fast, they still face three major challenges in real industrial-scale applications: irregular memory access patterns, high computational overhead, and painful parameter tuning. To address these issues, the VSAG vector database proposes three key optimization directions: memory optimization, automatic tuning, and computation acceleration. **Performance comparison:** Compared with the current industry-standard library HNSWlib, VSAG achieves up to a **4× speedup** at the same level of accuracy. **Deployment status:** VSAG has already been practically deployed and used in large-scale production workloads at Ant Group.

Keywords: VSAG library; memory optimization; distance computation; automatic tuning

1 VSAG 库简介 4
1.1 业务背景：为什么需要更强的向量搜索？ 4
1.2 现状分析：图搜索（Graph-based ANNS）的局限 4
1.3 VSAG 的四大贡献 5
① 内存访问：从“随机”变“连续” 5
② 自动调优：从“几天”缩短到“几次构建” 5
③ 计算加速：精细化管理 5
④ 实战结果 5
2 VSAG 库架构概览 6
2.1. 内存访问优化 6
1. 确定性访问（Deterministic Access）：软件预取 6
2. PRS 设计：部分冗余存储 7
3. 资源平衡：动态参数 δ 7
2.2 调参 8
1. 为什么调参这么难？（三层参数体系） 8
2. VSAG 的自动化调优方案 8
3.精细化算力管理：如何在毫秒级时间内计算万次高维距离？ 9
3 内存访问优化 10
核心矛盾：计算太快，内存太慢 10
3.1 软件预取（Software Prefetching） 11
1. 背景：高维向量带来的“沉重”负担 11
2. 核心技术：软件预取（Software Prefetching） 11
3. “逻辑上的连续”是什么意思 12
3.2 如何实现有效的预取 12
1. 传统预取的无效操作 12

2. 核心方案一：确定性访问 (Deterministic Access) 12
3. 核心方案二：步长预取 (Stride Prefetch) 13
Algorithm 1：确定性访问贪婪搜索 (核心逻辑拆解) 13
3.3 PRS（Partial Redundant Storage，部分冗余存储） 15
1. 软件预取的限制 15
2. 硬件预取的先天优势（Hardware-based Prefetch） 15
3. PRS 的核心操作：数据重排 15
4. 冗余率 delta：成本与性能的平衡 16
4 自动化调参 16
4.1 参数分类 16
1. 核心洞察：参数不是平等的 16
2. 三种参数的深层影响 17
4.2 如何让算法在任何硬件上都跑出最高速 17
1. 核心矛盾：硬件差异性 17
2. VSAG 的解法：三步自动化调优流程 18
4.3 利用 GBDT 模型实现查询级的‘减负’ 18
1. 核心洞察：查询的差距 18
2. VSAG 的解法：智能难度分类器 19
4.4 基于标签的索引自动调优 20
1. 核心矛盾：昂贵的“结构成本” 20
2. VSAG 的重大发现：层级包含关系 20
3. 核心实现：掩码/标签索引（Labeling Strategy） 20
4. 这种设计优势 21
5 距离计算加速 21
5.1 距离计算的成本 21
1. 核心数学模型：距离计算的成本公式 21

2. VSAG 的三大优化手段 22
5.2 利用 SIMD 与数学变换减少 CPU 浪费 23
1. 量化选择：为什么是 SQ 而不是 PQ 23
2. 数学变换：将欧式距离简化 23
5.3 如何用最少的成本消除量化误差 24
1. 核心矛盾：量化的“代价” 24
2. VSAG 的解法：选择性重排序 (Selective Re-ranking) 24

1 VSAG 库简介

1.1 业务背景：为什么需要更强的向量搜索

RAG（检索增强生成）：为了解决大模型的“幻觉”和信息滞后问题，必须通过向量搜索实时检索最新知识。非结构化数据爆发：支付宝的面部识别支付、Google 图片搜索、YouTube 视频搜索，本质上都是将图片/视频转化为向量进行匹配。

1.2 现状分析：图搜索（Graph-based ANNS）的局限

虽然 HNSW 和 VAMANA 是目前的主流，但蚂蚁集团在 GIST1M 数据集上的实验暴露了**三大性能瓶颈**：

痛点	实验数据 (基于 HNSW+SQ4)	后果
内存访问成本高	67.42% 的 L3 缓存缺失率；内存操作占总耗时 63.02%	CPU 大部分时间在“等数据”，效率极低。
参数调优成本高	最优参数比手动参数 QPS 高出 42.6%，但暴力搜索最优参数需 60+ 小时	调优太慢，甚至需要重新构建索引（耗时数天）。

距离计算开销大	即使用了 SQ4 量化，计算耗时仍占 26.12%	高维向量的数学运算依然是负担。

1.3 VSAG 的四大贡献

(1)内存访问：从“随机”变“连续”

- **预取（Pre-fetching）**：在程序还没用到邻居节点时，提前把它们加载到缓存。
- **邻居向量连续复制**：将经常一起访问的邻居向量在物理内存上存放在一起。
- **结果**：显著降低了 L3 缓存缺失率。

② 自动调优：从“几天”缩短到“几次构建”

- **智能覆盖**：涵盖了环境参数（预取深度）、索引参数（最大度数）和查询参数（候选池大小）。
- **效率飞跃**：以一个三维的空间为例，每个空间有 5 种选择，传统暴力搜索需要测试 53 = 125 种组合（即重建 125 次索引），VSAG 只需要 **2-3 倍**于单次索引构建的时间，就能达到相近的效果。

③ 计算加速：精细化管理

- **近似距离技术**：进一步优化了标量量化。
- **重排序（Re-ranking）策略**：先用低精度快速过滤，再用高精度精细校准，确保准确率（Recall）不掉队。

④ 实战结果

- 在 **1 亿级** 数据集上验证。
- 在相同准确率下，QPS（每秒查询数）最高是工业标准库 **HNSWlib 的 4 倍**。

2 VSAG 库架构概览

VSAG 框架核心架构概览

VSAG 的设计哲学是在搜索的每一个阶段都嵌入优化逻辑，具体可以分为以下四个部分：

- 底层存储（PRS 系统）：双重备份，软硬兼施
- 搜索过程：确定性访问贪婪搜索（Deterministic Access Greedy Search）
- 结果处理：选择性重排序（Selective Re-rank）
- 全局大脑：智能自调优（Smart Auto-tuner）

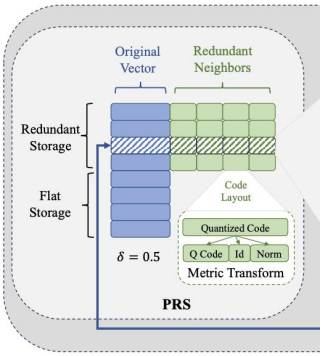
2.1. 内存访问优化

1. 确定性访问（Deterministic Access）：软件预取

核心思路： 既然图搜索的跳转是不可预测的，那我们就用“软件指令”强行让它变得可预测。

- **痛点：** 传统的图搜索像是在迷宫里随机跳跃，CPU 往往不知道下一个数据在哪，导致 CPU 闲置（等数据从内存传过来）。
- **VSAG 的做法：** 引入了**软件预取（Software Prefetching）**，通过 _mm_prefetch 指令实现。
    - **流水线重叠：** 在计算当前向量距离的同时，提前发出指令，“命令”内存把下一个邻居节点的数据加载到 L3 缓存。
    - **批处理与重排：** 并不是乱序预取，而是对访问序列进行**批处理和重新排序**，让预取操作更有序，避免无效的预取浪费带宽。

2. PRS 设计：部分冗余存储



核心思路： 空间换时间。既然去远处找邻居向量太慢，我就在每个节点旁边放一份邻居的“缩减版”。

- **硬件瓶颈：** 在生产环境（如 4 核 16G 机器）中，CPU 经常因为缓存缺失而空转。
- **PRS 的精妙之处：**
  - **冗余嵌入：** 在存储每一个节点（向量）时，不仅仅存它自己，还在它旁边**额外存储它所有邻居的压缩向量**（Compressed Neighbor Vectors）。
  - **硬件预取友好：** 因为邻居的压缩数据就在当前节点物理位置的附近，这能极大地触发**硬件预取（Hardware Prefetch）**。CPU 会认为你在读取连续数据，从而自动、高效地加载。
  - **高压缩比：** 通过量化技术（Quantization），这些冗余数据占用的空间并不大，保持了存储成本的可控。

3. 资源平衡：动态参数 δ

核心思路： 没有最好的配置，只有最适合业务场景的配置。

VSAG 引入了一个关键参数：**冗余率 δ**。

业务场景	δ 取值	策略说明	效果

高吞吐量（计算密集型）	**调高 δ**	增加冗余存储，减少 CPU 等待时间	最大化 QPS，减少 CPU 闲置
内存受限（低成本型）	**调低 δ**	减少冗余，精简索引体积	可以在更便宜、内存更小的机器上部署

2.2 调参

1. 为什么调参这么难（三层参数体系）

VSAG 首次将向量检索的参数分成了三类，每一类的调优成本和影响范围都不同：

参数级别	代表参数	影响因素	传统调优成本
环境级 (Environment)	预取步长	仅影响 QPS（速度）	**低**（改改代码配置就行）
查询级 (Query)	候选集大小	影响 QPS 和 召回率	**中**（需要针对不同数据分布测试）
索引级 (Index)	最大出度	影响 QPS、召回率、构建时间	**极高**（**必须重新构建索引**，耗时数天）

2. VSAG 的自动化调优方案

针对上述三类参数，VSAG 分别给出了对应的“药方”：

① 环境级：网格搜索（Grid Search）

- **做法：** 既然它只影响速度不影响精度，VSAG 会在当前硬件环境下，通过系统性的尝试（网格搜索）找到让 QPS 最高的那个配置。
- **价值：** 确保在不同型号的 CPU 上都能发挥最佳性能。

② 查询级：多粒度自适应（Adaptive Tuning）

- **做法：** VSAG 会根据查询的“难度”动态调整参数。
  - 如果这个查询向量在空间中比较孤立（容易找），就缩小搜索范围；
  - 如果周围向量很密集（难区分），就自动扩大搜索范围。
- **价值：** 保证每一条查询都能在精度和速度之间找到平衡点。

③ 索引级：基于掩码的“超大索引”技术

这是 VSAG 最具创新点的地方。

- **传统痛点：** 如果你想试试（最大出度）mc=32 和 mc=64 哪个好，你得建两次索引。
- **VSAG 的妙招：** 它引入了一种**基于掩码（Mask）的索引压缩技术**。
  - 它首先构建一个包含更多连接的“超级索引”。
  - **标签过滤：** 在搜索时，通过给边（Edge）打标签并进行动态过滤，来模拟不同参数下的索引结构。
  - **结论：** 它成功地将“构建时参数”变成了“查询时参数”。你只需要建一次索引，就能模拟出几十种不同参数组合的效果。

3.精细化算力管理：如何在毫秒级时间内计算万次高维距离？

1. 硬件红利：利用 SIMD 指令集加速

核心逻辑： 同样的时间，做更多的事情。

- **痛点：** 向量维度越高，计算欧式距离或内积的次数就越惊人。
- **VSAG 的做法：** 充分利用现代 CPU 的 AVX512 指令集。

- **性能对比：** 论文提到，在相同的指令架构下，处理 INT8（8 位整数）数据的速度是处理 FLOAT32（32 位浮点数）的 **4 倍**。
- **启发：** 尽量将数据从高精度的浮点数转化为低精度的整数进行运算，可以获得质的飞跃。

2. 选择性重排序（Selective Re-rank）：速度与精度的平衡

核心逻辑： 粗筛用低精度（快），精选用高精度（准）。

如果只是简单地把所有向量都量化为低精度（比如 SQ8 或 SQ4），虽然快，但准确率（Recall）会掉得很厉害。VSAG 采用了**选择性重排序**策略：

1. **第一步（粗筛）：** 在图搜索的大部分过程中，使用量化后的低精度编码进行距离计算。这能极快地排除掉那些明显不相关的向量。
2. **第二步（精滤）：** 仅针对最后最有希望的候选者（Visited results），调取原始的高精度向量进行重新排序（Re-rank）。
3. **效果：** 既享受了低精度计算带来的高吞吐量，又保持了和全量高精度计算几乎一致的准确率。

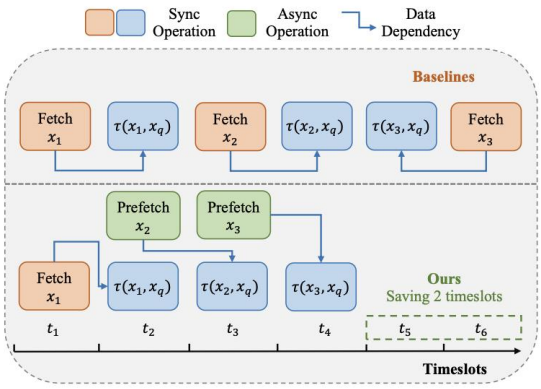
3 内存访问优化

核心矛盾：计算太快，内存太慢

在图搜索过程中，CPU 计算两个向量距离的速度极快，但从内存（RAM）里读取下一个邻居向量的速度相对很慢。

- **现状：** CPU 算完了一个距离，想算下一个，但数据还在从内存运往缓存（Cache）的路上。此时 CPU 只能“空转”等待。
- **VSAG 的解法：** 利用计算的“间隙”（Intervals）。在 CPU 忙着计算当前向量距离时，提前把后面要用的数据“拎”到缓存里。

3.1 软件预取（Software Prefetching）



1. 背景：高维向量带来的“沉重”负担

论文用一个具体的例子展示了为什么内存访问是瓶颈：

- **数据案例：** GIST1M 数据集，维度是 **960 维**，存储格式是 **float32**（每个数占 4 字节）。
- **算笔账：** 一个向量的大小就是 960 * 4 = 3,840 字节。
- **缓存限制：** CPU 从内存拿数据不是一个一个拿，而是按“缓存行”（Cache Line，通常是 **64 字节**）为单位。
- **后果：** CPU 每计算一个向量的距离，就得触发 3,840 / 64 = 60 次缓存行传输。

痛点： 在传统的搜索算法中，这 60 次传输是“被动”的。CPU 发现缓存里没有数据（Cache Miss），就停下工作，等内存把数据慢吞吞地搬过来。这就像厨师每炒一个菜都要停下来等外卖员送葱，严重浪费了 CPU 的算力。

2. 核心技术：软件预取（Software Prefetching）

VSAG 改变了这种“串行”模式，将其变成了“并行”：

- **逻辑转变：** 不再等“饿了”再点外卖。VSAG 在计算**当前**邻居向量的距离时，就已经发出指令（_mm_prefetch），让内存开始准备**下一个**邻居向量的数据。

- **异步执行：** 预取指令是**异步**的。这意味着 CPU 发出指令后，不需要停下来等数据到位，而是可以继续埋头计算当前的向量。

3. “逻辑上的连续”是什么意思

虽然图搜索在物理内存上的访问是“随机”的（跳来跳去），但通过提前预取，对于 CPU 核心来说：

1. 它在计算完向量 A 之后，向量 B 的数据已经**提前**躺在 L3 缓存里了。
2. 对 CPU 而言，它感觉自己一直在连续不断地处理数据，没有遇到任何阻塞。
3. **效果：** 这种技术在逻辑上把碎片化的随机访问“缝合”成了一条顺滑的数据流。

3.2 如何实现有效的预取

1. 传统预取的无效操作

论文指出，简单的预取策略存在两个致命缺陷：

- **重复预取：** 在图搜索中，不同的点可能有共同的邻居。如果程序不加判断地预取所有邻居，很多已经访问过（甚至已经在缓存里）的点会被重复加载，白白浪费带宽。
- **时机失控：** 预取太早，数据还没用就被挤出缓存了；预取太晚，计算时数据还没到，CPU 还是要等。

2. 核心方案一：确定性访问 (Deterministic Access)

核心逻辑：先过滤，后加载。

VSAG 改变了“一边看邻居一边预取”的习惯，转而采用**批处理模式**：

1. **批量检查：** 先把当前节点的所有邻居拎出来，检查一遍它们的“访问状态”。
2. **精准过滤：** 把那些已经访问过的邻居剔除掉。
3. **集中预取：** 只对剩下的、**真正需要计算距离**的“生面孔”发起预取。

效果： 确保每一次发出的预取指令都是有意义的，彻底消灭了冗余的内存操作。

3. 核心方案二：步长预取 (Stride Prefetch)

核心逻辑：寻找预取的“黄金窗口”。

预取是一个**异步**的过程，没有回调函数告诉你“数据已送达”。因此，时机全靠参数“预取步长”来控制。

- **参数“预取步长”的含义：** 它定义了在发起下一次预取之前，CPU 要执行多少步计算。
- **动态平衡：**
  - 如果计算很快（CPU 强），**预取步长** 就要调小，让预取更频繁、更超前。
  - 如果内存很慢，**预取步长** 可能需要调整以给内存留出足够的搬运时间。
- **目标：** 让数据恰好在 CPU 伸手要的那一刻，精准地出现在缓存里。

Algorithm 1：确定性访问贪婪搜索 (核心逻辑拆解)

Algorithm 1: Deterministic Access Greedy Search

Input: graph G, labels L, base dataset D, initial nodes I, query point x_q, low- and high- precision distance functions τ_l and τ_h, search parameters k, ef_s, m_s, α_s, ω, ν

Output: $ANN_k(x_q)$ and their high-precision distances T

```
candidate pool C ← maximum-heap with size of ef_s
visited set V ← ∅
insert (x_i, τ_l(x_i, x_q)), ∀x_i ∈ I into C
while C has un-expended nodes do
    x_i ← closest un-expended nodes in C
    N ← empty list
    for j ∈ G_i do
        // Only retrieve Id(i.e., x_j.id = j)
        if j ∉ V and L_j ≤ α_s and |N| < m_s then
            N ← N ∪ {j}
            V ← V ∪ {j}
    for k ∈ [0, min(ω, |N|)) do
        prefetch ν cache lines start from D_{N_k}
    for k ∈ [0, |N|) do
        if k + ω < |N| then
            prefetch ν cache lines start from D_{N_{k+ω}}
        x_j ← D_{N_k} // Memory Access
        insert (x_j, τ_l(x_j, x_q)) into C and keep |C| ≤ ef_s
ANN_k(x_q), T ← selective re-rank C with τ_l and τ_h
return ANN_k(x_q), T
```

为了方便理解，把这个算法分为三个阶段：**筛选、排队预取、精准计算**。

1. 准备阶段 (初始化)

- C (**候选池**)：一个大顶堆，用来存目前离查询点 xq 最近的候选向量。
- V (**访问名单**)：记录哪些点已经看过了，防止“原地打转”。

2. 核心循环：如何“聪明地”找邻居

算法的核心在于第 7 行到第 17 行，这里体现了 VSAG 的独特优化：

- **第一步：精准筛选** (Lines 7-10)
  - 在计算距离之前，先扫描当前节点的所有邻居。
  - **过滤条件：** 只有**没访问过** 且 **符合索引标签** 的邻居才会被放入待计算列表 N。
  - *这就是“确定性访问”，不盲目预取，只针对真正要算的点。*
- **第二步：启动预取** (Lines 11-12)
  - 在开始正式计算前，先一口气把前 omega（预取步长）个邻居的数据发往缓存。
- **第三步：流水线作业** (Lines 13-17)
  - 当 CPU 正在处理第 k 个邻居的距离计算时，代码会自动发出指令预取第 k + omega 个邻居的数据。
  - **效果：** 计算和内存传输完全重叠。当你算完这一个，下一个的数据正好到达了 L3 缓存。

3. 收尾：精益求精 (Line 18)

- Selective Re-rank (**选择性重排序**)：搜索结束后，堆里剩下的是一堆用“低精度”算出来的候选者。VSAG 最后会用“高精度”再复核一遍，确保给用户的 Top-K 结果万无一失。

3.3 PRS（Partial Redundant Storage，部分冗余存储）

1. 软件预取的限制

虽然 Algorithm 1 里的软件预取很厉害，但它有三个绕不过去的坑：

1. **只是个建议：**_mm_prefetch 指令告诉 CPU “帮我拿数据”，但 CPU 如果现在很忙，可以完全**不理会**这个建议。
2. **缓存污染：** 在多核环境下，如果预取太激进，会把别人正在用的有用数据从 L3 缓存里挤出去。
3. **自身开销：** 软件预取指令本身也是要消耗 CPU 周期的。

2. 硬件预取的先天优势（Hardware-based Prefetch）

CPU 内部其实自带了一个硬件预取器，它会观察你的访问习惯：

- **它喜欢顺序访问（Sequential Access）：** 如果你一直在读连续的内存（像数组一样），它能秒懂并提前帮你把后面的数据搬过来。
- **它讨厌随机访问（Random Access）：** 传统的图算法（如 HNSW）是在内存里到处乱跳，硬件预取器无法发挥作用。

3. PRS 的核心操作：数据重排

核心逻辑：既然图访问是随机的，那我就把数据存成连续的。

VSAG 的做法：**冗余存储（Redundancy）**。

- **做法：** 在存储每一个节点时，不光存它自己的向量，还把它的所有邻居的向量（通常是压缩后的）**物理上紧挨着存放在这个节点后面**。
- **效果：** 当算法要访问一个节点的邻居时，在内存地址上，它是在读一段**连续**的空间。
- **结果：** 这一改动瞬间“激活”了 CPU 内部的硬件预取器。它会认为你在读连续数组，从而自动、高速地把邻居数据搬进缓存。

4. 冗余率 delta：成本与性能的平衡

冗余存储虽然快，但费内存。VSAG 引入了一个可调参数 **δ** 来让用户根据预算选方案：

方案	冗余率 δ	优点	缺点	适用场景
极致性能	**δ** = 1	硬件预取满血运行，CPU 几乎不空转	索引体积翻倍	预算充足、追求极致响应速度
极致省钱	**δ** = 0	内存占用最小	依赖软件预取，速度相对较慢	内存受限、对成本极度敏感
平衡模式	0 < **δ** < 1	兼顾性能与成本	需要调优	工业界大多数生产环境

4 自动化调参

4.1 参数分类

1. 核心洞察：参数不是平等的

论文指出，向量搜索之所以难以优化，是因为参数太多且性质各异。VSAG 首次将这些参数分成了三类，并定义了它们的“调优难度（Hardness）”：

参数级别	典型参数	影响范围	优化目标	调优难度
环境级 (ELP)	预取步长 、深度	仅影响 QPS（速度）	**单目标**：效率最大化	**低**（不影响准确率）
查询级 (QLP)	搜索池大小、重排策略	影响 QPS 和 召回率	**多目标**：权衡速度与精度	**中**（无需重构索引）

索引级 (ILP)	最大出度 、构建参数	影响 QPS、召回率、构建时间	**极致挑战**：权衡三者	**极高**（**必须重构索引**）

2. 三种参数的深层影响

① 环境级参数 (ELPs)：硬件的调速器

- **本质**：它们不改变算法找邻居的逻辑，只改变“拿数据”的速度。
- **特点**：即使设错了，顶多是搜索变慢，但最后找到的结果（召回率）是一模一样的。
- **调优逻辑**：只需要针对特定的 CPU、内存环境，找到能让指令跑得最顺的配置即可。

② 查询级参数 (QLPs)：精度与速度的平衡参数

- **本质**：决定了你在图里翻找得“深不深”。
- **特点**：这是一个典型的 Trade-off。找得越深（ef_s 越大），准确率越高，但耗时也越长。
- **调优逻辑**：需要根据具体的查询数据分布，动态寻找那个“性价比”最高的点。

③ 索引级参数 (ILPs)：数据库的关键参数

- **本质**：决定了索引图是怎么连线的。
- **痛点**：以前你想测试把每个点的邻居上限从 32 改成 64，你必须花几个小时甚至几天时间**把整个索引重新建一遍**。
- **调优逻辑**：这是工业界最想解决的问题，VSAG 后面会通过“掩码索引”技术来攻克它。

4.2 如何让算法在任何硬件上都跑出最高速

1. 核心矛盾：硬件差异性

论文指出，两个关键参数“预取步长”和“预取深度”在不同的机器上，最优值完全不同：

- **计算速度 vs 内存延迟：** 如果 CPU 算得极快（比如最新的 Intel 至强处理器），但内存搬运慢，你就得把 预取步长 调小，让数据“提早出发”。
- **硬件原生偏好：** 不同的 CPU 架构对一次性搬运多少个缓存行有不同的最优值，这还跟向量本身的维度有关。

痛点： 靠人工去每一台服务器上试出最优的预取步长和预取深度简直是运维的噩梦。

2. VSAG 的解法：三步自动化调优流程

由于 ELPs **只影响速度、不影响精度**，VSAG 可以非常放心地在后台开启“自动试错模式”：

第一步：建立参数网格 (Grid Search)

系统会自动列出预取步长和预取深度的所有可能组合。例如，预取步长取 [1, 2, 4, 8]，预取深度取 [1, 2, 4]，组合成一个测试矩阵。

第二步：小样本实测 (Sampled Evaluation)

VSAG 不会拿着全部亿级数据去试，而是从原始数据集中抽取一小部分样本（Sampled vectors），在真实的环境中运行这些样本查询。

第三步：择优录取 (Selection)

系统会自动对比不同组合下的 QPS（每秒查询数）。

- **目标：** 选出那个能让 QPS 达到峰值，且运行最稳定的配置。

4.3 利用 GBDT 模型实现查询级的‘减负’

1. 核心洞察：查询的差距

VSAG 团队发现了一个非常有意思的现象（高度偏态分布）：

- **99% 的查询** 都是“送分题”：只需要很小的搜索参数（比如很小的候选集大小）就能达到目标召回率。

- **1% 的查询** 是“压轴题”：需要非常大的搜索空间才能找准。

痛点： 传统的做法是“一刀切”，为了保住那 1% 难题的准确率，给所有查询都设置了很高的参数。结果就是 99% 的简单查询都在做**无用功**，白白浪费了 3-5 倍的性能。

2. VSAG 的解法：智能难度分类器

VSAG 引入了一个 GBDT（**梯度提升决策树**）模型，像一个“监考老师”一样实时判断当前的查询难不难。

① 它是如何判断的？（特征工程）

模型会根据搜索过程中的实时状态来打分：

- **扫描点数：** 已经看了多少个向量了？
- **候选者分布：** 目前最接近的那 5 个候选人，它们彼此之间离得远吗？（如果离得很近，说明竞争激烈，很难选）。
- **进度变化：** 最近几次找到的更近结果，距离缩小的速度快吗？（如果进步变慢了，可能快到头了）。
- **距离差：** 当前最优解和候选解之间的相对差距。

② 它是如何“减负”的？（早期停止机制）

1. **先给高配：** 初始给一个较大的搜索空间，如 ef_s（候选集大小）= 300。
2. **中途评估：** 在图里跳了几步后，GBDT 模型立刻介入。
3. **果断止损：** 如果模型判断这是一个“简单查询”，它会立即把 ef_s 调低。
4. **效果：** 防止简单的查询在图里漫无目的地瞎逛，从而节省大量计算资源。

4.4 基于标签的索引自动调优

1. 核心矛盾：昂贵的“结构成本”

论文指出，索引级参数（ILP，如最大出度）决定了图的物理结构。

- **痛点：** 以前你要想从 m_c（最大出度）=32 调到 m_c=64，必须推倒重来，花几个小时重新建一遍索引。这种“非线性”的调参方式，让寻找最优结构变得几乎不可能。

2. VSAG 的重大发现：层级包含关系

VSAG 团队提出了一个定理（Theorem 4.1）：

- **内容：** 在同样的初始化条件下，用较小 m_c 构建出来的图，其实是较大 m_c 构建出来的图的**真子集**。

3. 核心实现：掩码/标签索引（Labeling Strategy）

基于上述发现，VSAG 彻底改变了索引的生命周期，从“构建 -> 搜索”变成了“**构建 -> 调优 -> 搜索**”。

① “超级索引”的构建

不再用固定的 m_c（最大出度）构建索引，而是用一个很宽松的上限（比如 m_c=128）直接建一个“超级索引”。

② 给边“贴标签”（Labeling）

这是最关键的一步：在建索引的过程中，VSAG 会记录每一条边是在 m_c 等于多少的时候被加入的。

- 例如：边 A 是在 m_c=16 时就存在的，标记为“等级 1”；边 B 是在 m_c=64 时才补上的，标记为“等级 4”。

③ 搜索时的“逻辑过滤”

当用户在调优阶段想测试 m_c=32 的效果时：

- VSAG **不需要重构索引**。
- 它在搜索时直接加上一个过滤条件：**“只走那些标签 <=2 的边”**。
- **结果：** 在同一个物理索引上，逻辑上完美模拟出了一个小于等于 m_c 索引的表现。

4. 这种设计优势

维度	传统方法	VSAG (Labeling)
调参速度	极慢（重构耗时数小时）	**秒级**（改个过滤数字即可）
存储开销	存 5 套参数需 5 倍空间	**1 倍空间**（存一个超级索引即可）
灵活性	静态，一旦建好无法更改	**动态**，可根据实时负载随时切换图结构

5 距离计算加速

5.1 距离计算的成本

1. 核心数学模型：距离计算的成本公式

VSAG 将搜索过程中的计算成本拆解为一个清晰的数学公式：

Cost = (n_lp * t_lp) + (n_hp * t_hp)

- lp (Low Precision)：低精度阶段（在大规模图遍历时使用）。
- hp (High Precision)：高精度阶段（在最后的重排序阶段使用）。

- **n**：计算次数。
- **t**：单次计算的时间。

VSAG 的策略： 因为 n_lp（低精度搜索步数）和 t_hp（高精度浮点数计算耗时）很难再压榨，所以 VSAG 把核心优化点放在了 **t_lp（降低单次低精度计算耗时）** 和 **n_hp（减少高精度计算次数）** 上。

2. VSAG 的三大优化手段

① 极速低精度计算 (t_lp)

- **量化 + SIMD**：通过标量量化（SQ）将 32 位浮点数压缩为 8 位或更低的整数。利用现代 CPU 的 AVX512 指令集，一个指令周期内处理的数据量比浮点数多出数倍。
- **内存对齐**：配合之前提到的 PRS 存储，确保数据读取和计算在流水线上是无缝衔接的。

② 提高量化精度以减少候选量 (n_hp)

- **痛点**：如果量化得太粗糙（比如精度损失太大），为了保证召回率，算法不得不扩大搜索范围（导致 n_lp 增加）。
- **优化**：VSAG 通过参数优化（如 Appendix G 提到的量化参数动态调整），在保持低精度的同时尽可能保留原始向量的特征。这样不需要搜索太多点就能找到目标，从而间接减少了后续需要高精度复核的工作量。

③ 选择性重排序 (n_hp)

- **动态阈值**：传统的 ANNS 会在搜索结束后对所有 ef_s（候选集）个候选点进行重排。
- **VSAG 的创新**：它建立了一个“动态平衡”。只对那些“最有希望”的子集进行高精度验证。这个子集的规模是随候选总数对数级（Logarithmically）增长的，而不是线性增长。
- **结果**：极大地压缩了昂贵的 t_hp 计算次数。

5.2 利用 SIMD 与数学变换减少 CPU 浪费

1. 量化选择：为什么是 SQ 而不是 PQ

论文对比了两种主流的向量压缩技术：

- **PQ (乘积量化)**：虽然压缩率极高，但它依赖“查表（Lookup Table）”。在图搜索这种随机访问模式下，频繁查表会导致 CPU 缓存失效，反而浪费了 SIMD 指令的带宽。
- **SQ (标量量化)**：VSAG 选择了 SQ（如将 32 位浮点数转为 8 位或 4 位整数）。
  - **优点**：不需要查表，直接在压缩数据上进行并行计算。
  - **硬件红利**：在 AVX512 指令下，如果将数据压缩到 128 位，处理速度直接提升 **4 倍**。

2. 数学变换：将欧式距离简化

传统的欧式距离计算包含“减法、平方、累加”，非常繁琐。VSAG 利用数学分解将其简化：

欧式距离公式变形：

（x_b - x_q）^2 = x_b^2 + x_q^2 - 2 x_b * x_q

- **x_b^2 (基础向量模长平方)**：在**构建索引**时就已经算好了，存在内存里。
- **x_q^2 (查询向量模长平方)**：在**搜索开始前**算一次即可。
- **x_b * x_q$ (点积)**：这是搜索过程中唯一需要实时计算的部分。

优化结果：

通过这种变换，原本复杂的欧式距离计算被简化成了 **“一次点积运算”**。

- 省去了大量的减法（Subtraction）指令。
- **收益:**每次计算节省一个 CPU 时钟周期。在亿级数据的检索中，这种“省一个周期”的累积效应是非常惊人的。

5.3 如何用最少的成本消除量化误差

1. 核心矛盾：量化的“代价”

- **收益**：通过 SQ 量化（如 FLOAT32 -> INT8），计算速度提升了数倍，内存带宽压力也减小了。
- **痛点（量化误差）**：压缩数据会导致距离计算不准。原本排在第 1 名的向量，量化后可能变成了第 10 名。如果直接返回低精度结果，召回率（Recall）会大幅下降。

2. VSAG 的解法：选择性重排序 (Selective Re-ranking)

传统的做法是对候选集（Candidate set）里的所有向量进行高精度计算，但这太慢了。VSAG 采用了更聪明的“分治”策略：

① 什么是“选择性”？

VSAG 不会盲目重排所有点，而是只挑选那些“由于量化误差可能导致排名反转”的候选者。

- **逻辑**：如果一个向量在低精度下的距离已经远远超过了第 k 名，那么即使有量化误差，它也很难逆袭进入前 k。我们只需要关注那些距离非常接近的“种子选手”。

② 动态重排范围（DDC 方案）

重排多少个点才算合适？固定值（比如总是重排前 100 个）在不同查询下表现很差。

- VSAG 引入了 DDC（Dynamic Distance Correlation）方案：
  - 它会分析当前查询的**误差-距离相关性**。
  - **自适应调整**：根据查询的特征、量化误差的分布以及用户要求的 k 值，动态计算出一个最优的重排窗口。
  - **效果**：难题多重排，简单题少重排，将高精度计算的次数（t_hp）压减到极致。

参考文献

[1] Alipay. 2025. Ant Group. https://www.antgroup.com.

[2] [2] Alipay. 2025. Face Recognition. https://open.alipay.com/api/detail?code=I1080300001000043632.

[3] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In Proceedings of the thirtieth annual ACM symposium on Theory of computing. 604 – 613.

[4] Intel. 2025. AVX512. https://www.intel.com/content/www/us/en/products/docs/ accelerator-engines/what-is-intel-avx-512.html.

[5] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. Advances in Neural Information Processing Systems 32 (2019).